```
  timing = Benchmark.measure do
    result = block.()
  end
  puts "It took: #{timing}"
  result
end
```

The next time you start irb, you'll be able to use this method to get timings:

```
irb(main):001:0> time { 1_000_000.times { "cat".upcase } }
It took:   0.106198   0.000795   0.106993 (  0.106986)
=> 1000000
```

Another common thing to do is reopen the Object class and add some diagnostic methods. Here's one that looks up the source location of a method (via https://medium.com/simply-dev/do-more-with-rails-console-by-configuring-irbrc-e5c25284305d).

```
class Object
  def sl(method_name)
    self.method(method_name).source_location rescue "#{method_name} not found"
  end
end
```

We'll see in Chapter 17, Debugging Ruby, on page 291 that the Pry tool also provides great support for looking up source locations.

You can configure irb to remember the commands you enter between sessions. Simply add the following to your .irbrc file, where the number indicates the amount of commands you want to save:

```
IRB.conf[:SAVE_HISTORY] = 50
```

Interactive Configuration

Most configuration values are also available while you're running irb. The list in irb Configuration Options, on page 285 shows these values as conf._xxx_. For example, to change your prompt back to SIMPLE, you could use the following:

```
irb(main):001:0* 1 +
irb(main):002:0> 2
=> 3
irb(main):003:0> conf.prompt_mode = :SIMPLE
=> :SIMPLE
?> 1 +
>> 2
=> 3
>>
```

irb Configuration Options

In the descriptions that follow, a label of the form :XXX signifies a key used in the IRB.conf hash in an initialization file, and conf.xxx signifies a value that can be set interactively.

:AUTO_INDENT / auto_indent_mode
: If true, irb will indent nested structures as you type them. The default is true.